


```

Enter the string to encode: aaaaaa
Enc(flag + txt )=
62064340711725050200219707729536152745043346180158508935012083119153637266868485155539
86827818840777322925424926979762114547563428575373316823436264390618726357037373547355
22495518690660451051105775094085936857731559313739549413555574879571339569391724383078
71657636214566519987125589046003140308292402011879635452242523595938292425145276083901
45873952760068476345341078313609446375182958802547847816648040378023443406558254598065
98187482240269757668153144893737506815052905722151968747809339143544153387022815815156
40005225941594046477076718367266639927968675685422316740413333458470982156584142436566
74504238331618
Enter the Selection from below:
1.Print PublicKey
2.Encrypt
0.Exit
0
BYE

```

So it works as following

- It gives out rsa public key, from which we can get N and e
- Gives out rsa encryption of flag + input

Seems like a nice application of [Franklin reiter Related message attack](#), which is

If we have two linearly related messages, and their corresponding ciphertexts and a small exponent e, we can figure out the messages by factoring the polynomial

Plan of actions go as follows:-

- Get public key, parse out N and e
- Get the encryption of flag + ""
- Get the encryption of flag + "a" a small message
- So we have two messages, int(flag) and 256*int(flag)+ord('a')

```

from pwn import remote
from Crypto.PublicKey import RSA
from base64 import b64decode
import re

HOST, PORT = "crypto.zh3r0.ml", 9841
REM = remote(HOST, PORT)

data = REM.recvuntil(b'0.Exit')
print(data.decode())

REM.sendline(b'l')
pub_key = REM.recvuntil(b'-----END PUBLIC KEY-----\n').strip()

rsa = RSA.importKey(b64decode(pub_key.split(b'\n-----END PUBLIC KEY-----\n')
[0].split(b'-----BEGIN PUBLIC KEY-----')[1]))
print(pub_key.decode())

N = rsa.n
E = rsa.e

```

```

REM.sendline(b'2')
REM.sendline()
REM.sendline(b'2')
REM.sendline(b'a')
REM.sendline(b'0')
data = REM.recvuntil(b'BYE')
print(data.decode())

C1, C2 = map(int, re.findall(b'(\d+)\n',data))

```

This part of the calculation I did it in sage

```

diff = ord('a')
R.<X> = Zmod(n)[[]]
f1 = X^3 - C1
f2 = (256*X + r)^3 - C2
def my_gcd(a, b):
    return a.monic() if b == 0 else my_gcd(b, a % b)
mint=-my_gcd(f1, f2).coefficients()[0]
print(bytes.fromhex(hex(mint)[2:]).decode())

# mint =
84490476860330212673060501801652621134953572462322324101674679636525065183282486185148
35453246148791206772560830219926411917729078654598549324669241020566018943465858086603
02373052129404619640702008492686731422271189803268079021294310388505025655889051036752
10898161818686994591193583661769377667810413989357162402550803291023005993338743213389
843385122094
# "RSA is secure and all but the only thing I want to say is
zh3r0{Hey_y0u_Sh0u1dn't_S3nd_r3l4ted_m3ssag3s_0r_h4v3_shot_p4ddings_wh3n_e_1s_sm411!!!
!!}.

```

zh3r0–Hey`y0u`Sh0u1dn't`S3nd`r3l4ted`m3ssag3s`0r`h4v3`shot`p4ddings`wh3n`e`1s`sm411!!!!
”

Interactive controls are omitted from this PDF. See the canonical HTML version.